EduGenie: Google Gemini Powered Learning Assistant

Project Description:

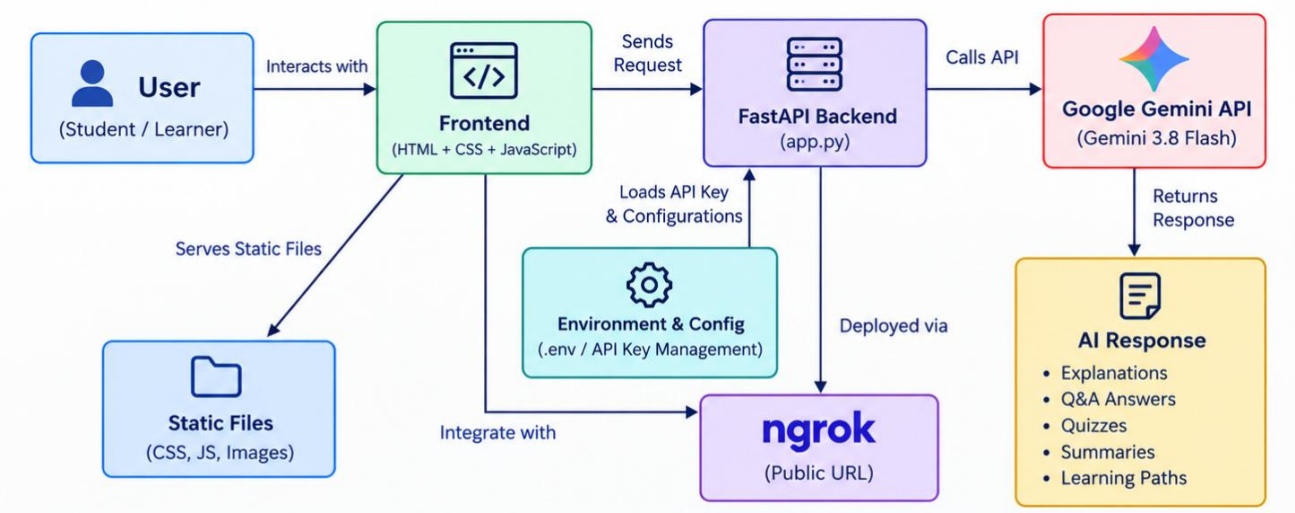
EduGenie is a lightweight, AI-powered educational assistant designed to simplify learning through generative AI. Targeted at students of all academic levels, EduGenie offers smart question answering, simplified explanations of complex concepts, automated quiz generation, personalized learning recommendations, and passage summarization.

Built with a FastAPI backend and an HTML + CSS frontend, the system leverages cloud-based AI models alongside lightweight local models for high efficiency on devices like Apple Silicon (Mac M1), ensuring broad accessibility for learners and developers alike.

Scenarios

Scenario 1: A student curious about geographical facts asks EduGenie, "Which is the largest ocean?" to receive a direct, concise response.
Scenario 2: A student evaluating her understanding of "The Pythagoras Theorem" clicks "Get Answer" to attempt auto-generated multiple-choice questions.
Scenario 3: A self-learner exploring SQL requests a structured, step-by-step learning path from beginner to advanced levels.

Technical Architecture:



Description: EduGenie uses a modular three-tier architecture to deliver AI-powered learning assistance. The frontend provides a responsive user interface built with HTML, CSS and JavaScript. The FastAPI backend handles user requests, processes the data, and communicates with the Google Gemini API (Gemini 3.8 Flash) for intelligent responses. The application uses environment variables (env) for secure API key management and is deployed publicly using Ngrok for easy access.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- Python 3.10+
- FastAPI framework knowledge.
- HTML and CSS knowledge for the frontend.
- Google Gemini API key.
- Uvicorn (ASGI server).
- Jinja2 for HTML templating.
- Python programming proficiency.
- A suitable development environment.

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Google Gemini is selected as the AI model for EduGenie because it can understand natural-language questions and generate useful educational responses.
- **Activity 1.2:** The system follows a frontend-backend-AI architecture.
- **Activity 1.3:** Architecture Flow
  User → HTML/CSS/JavaScript → FastAPI Backend → Google Gemini API → FastAPI Backend → User.

Milestone 2: Core Functionalities Development

EduGenie provides the following major functionalities:

- **Activity 2.1:** Q&A

  Provides answers to students' questions.
- **Activity 2.2:** Explain

  Explains difficult topics in a simple format.
- **Activity 2.3:** Quiz

  Generates questions for practice.
- **Activity 2.4:** Summary

  Creates short summaries from provided content.

Milestone 3: App.py Development

- **Activity 3.1:** The app.py file acts as the main backend file for the EduGenie application. It is developed using Python and FastAPI to handle user requests and connect the frontend with the Google Gemini AI service.

Milestone 4: backend Development

- **Activity 4.1:** FastAPI – Handles API requests and application routing.
- **Activity 4.2:** Google Gemini API – Generates AI-powered responses.
- **Activity 4.3:** Uvicorn – Runs the FastAPI application server.
- **Activity 4.4:** Jinja2 – Connects the frontend templates with the backend.

Milestone 5: Deployment

- **Activity 5.1:** After completing development and testing, the application can be run using the FastAPI server.
- **Activity 5.2:**The required environment variables are configured using a .env file.

- **Activity 5.3:** Example:

  GEMINI_API_KEY=your_api_key
- **Activity 5.4:** The application is tested locally before deployment.

Milestone 6: Conclusion

The development milestones of the EduGenie project were completed successfully. The project architecture, Google Gemini AI integration, core learning functionalities, backend development using FastAPI, and frontend development were implemented as planned.

The app.py backend connects the user interface with the Google Gemini API and handles the different learning features such as question answering, topic explanation, quiz generation, summarization, and learning recommendations.

Overall, these milestones helped build EduGenie as a simple, interactive, and AI-powered learning assistant for students.

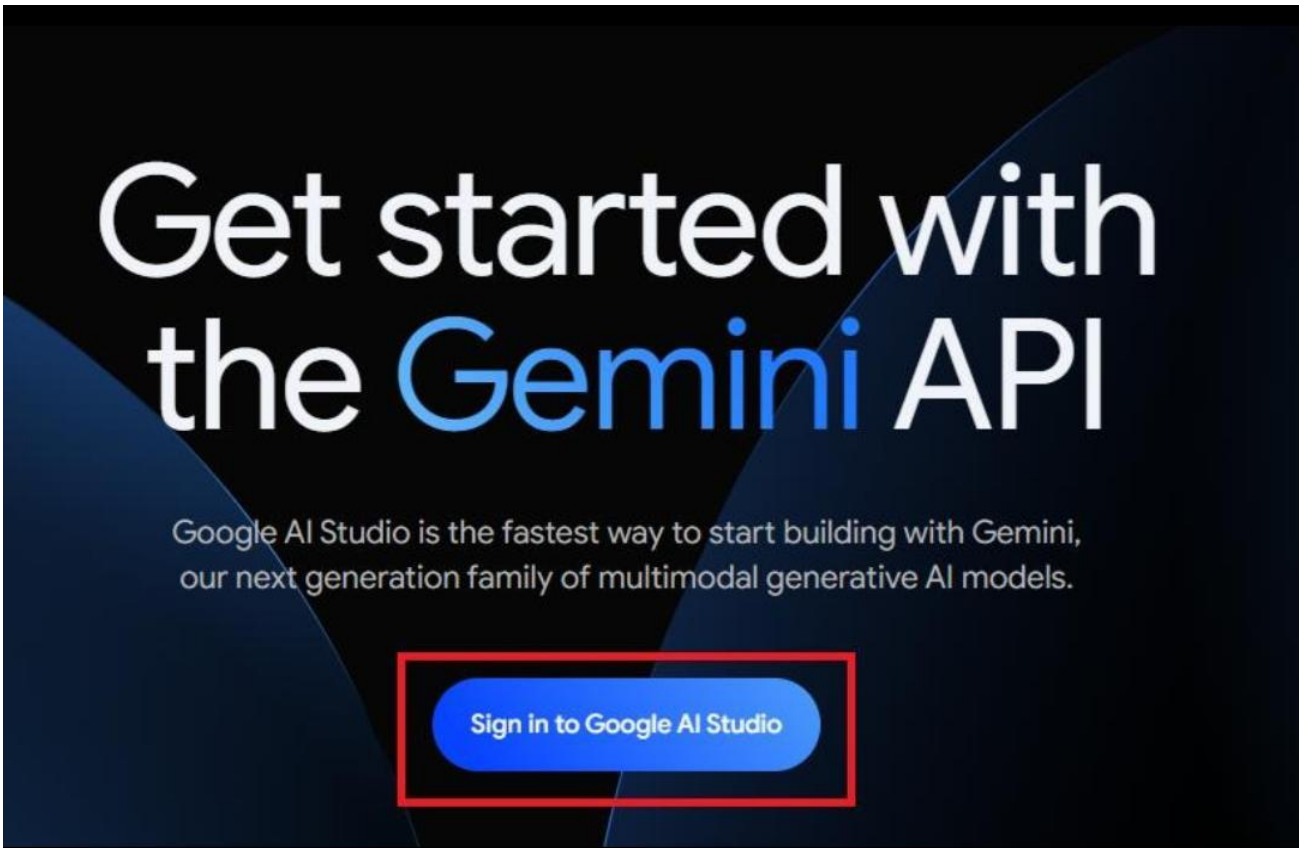
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model for the EduGenie learning assistant. This involves understanding the capabilities of Google Gemini and designing an architecture that supports question answering, topic explanations, quiz generation, summarization, and learning recommendations.

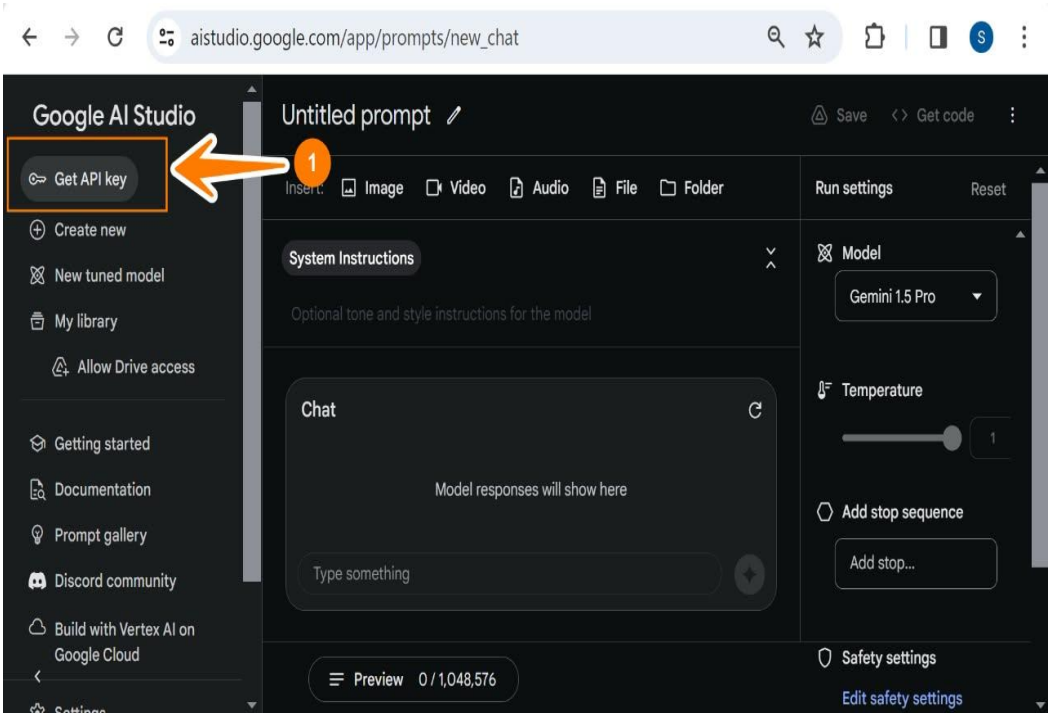
Activity 1.1: Select the Google Gemini Model

Before developing the application, a suitable Google Gemini model is selected based on its ability to understand student queries and generate useful educational responses.

Step 1 — Create a Google AI Studio Account:
https://aistudio.google.com and sign in using your Google account. Google AI Studio provides access to Gemini models and API key management.

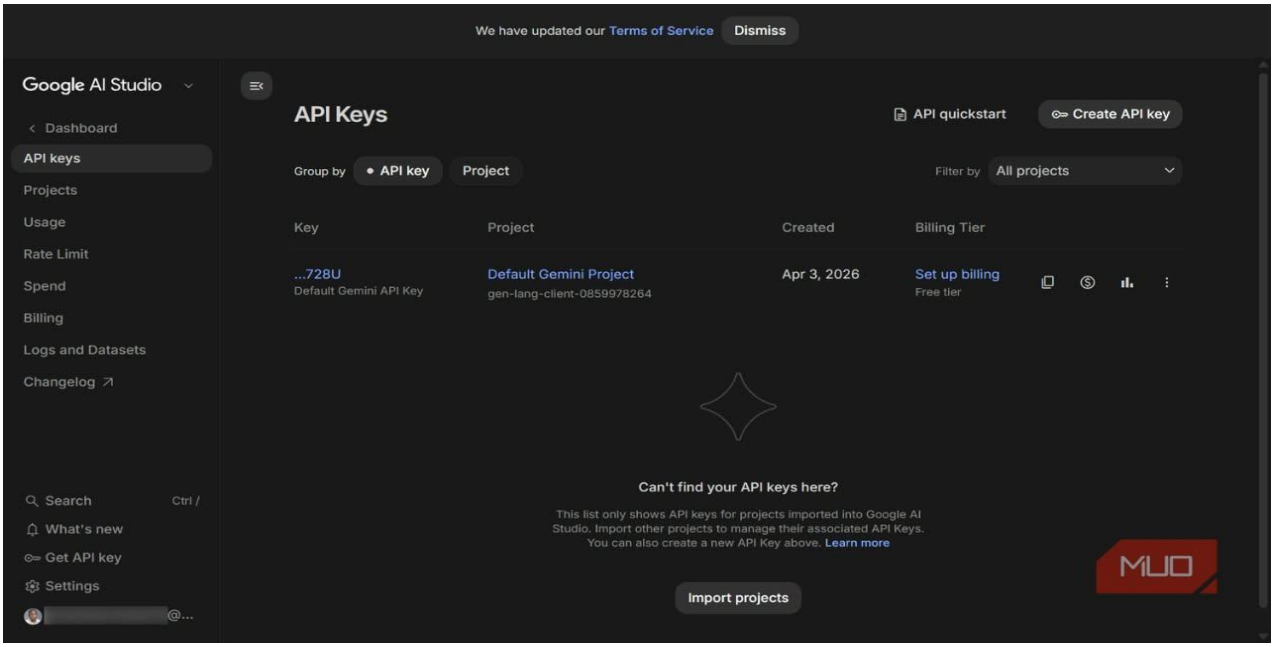


Step 2 — Navigate to API Keys:
After logging in, open the API Keys section in Google AI Studio. From there, you can create or view Gemini API keys.



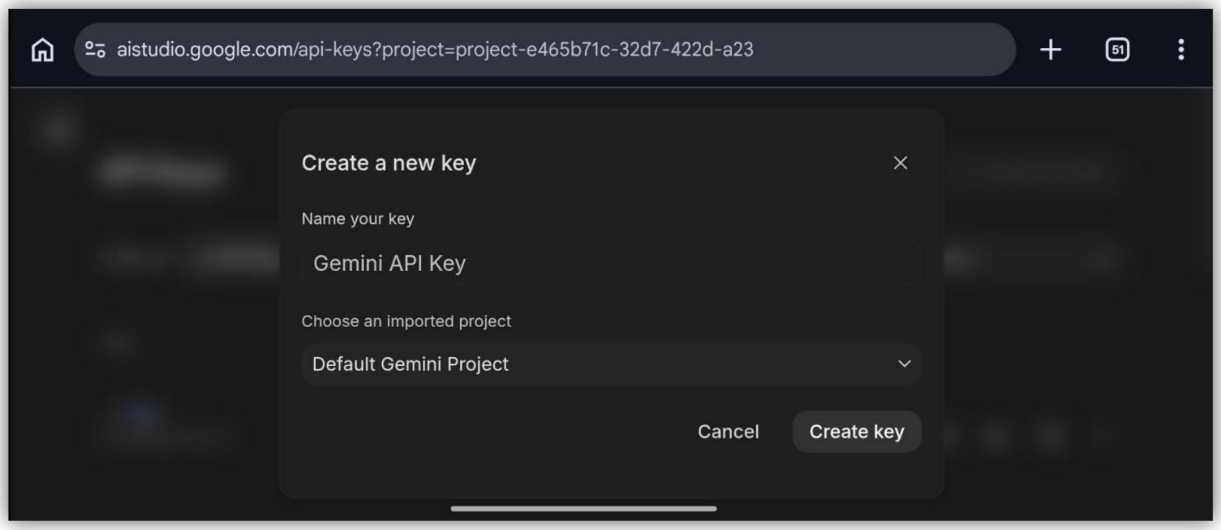
Step 3 — Create a Gemini API Key:

Click Create API key and select the required Google Cloud project when prompted. Google AI Studio can automatically create a project and API key for new users.



Step 4 — Copy and Secure the API Key:

Copy the generated API key and store it securely. For the EduGenie project, it can be configured through an environment variable such as GEMINI_API_KEY.



Step 5 — Add the Gemini API Key

Create a .env file in the root folder of your EduGenie project and add:

GEMINI_API_KEY=your_gemini_api_key_here

Step 6 — Verify the API Key is Loaded

The application loads the Gemini API key automatically using python-dotenv:

```
from dotenv import load_dotenv
import os
load_dotenv()
api_key = os.getenv("GEMINI_API_KEY")
```

The Gemini client can then be initialized using the API key.

Important: Never share or upload your actual API key. Add .env to .gitignore so the key is not accidentally committed to a public repository:

.env

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things researched to select the best AI model for the EduGenie project:

Understand the Project Requirements: Review the requirements of EduGenie, including Question Answering, concept explanation, quiz generation, text summarization, and personalized learning recommendations.

Explore Google Gemini: Study the Google Gemini API and its capabilities for educational question answering, summarization, quiz generation, and learning recommendations.
Evaluate Model Performance: Compare the model based on response quality, accuracy, generation speed, educational usefulness, and ability to understand different types of student queries.

Select the Generative AI Model: Select Google Gemini as the primary cloud-based generative AI model because it provides high-quality responses and supports multiple educational tasks through API integration.

Activity 1.3: Define the Architecture of the Application

Draft an Architectural Diagram: Create a visual representation of the EduGenie architecture, including the Frontend (HTML, CSS, JavaScript), FastAPI Backend, EduGenie Modules, and Google Gemini API.
Detail Frontend Functionality: Design the web interface where students can select a task such as Q&A, Explanation, Quiz, Summary, or Learning Recommendations, enter their input, and receive the generated result.
Outline Backend Responsibilities: Specify how the FastAPI backend receives user requests, validates the input, calls the appropriate EduGenie module, communicates with the Google Gemini API, and returns the result to the frontend.
Describe AI Integration Points: Define how the backend integrates with the Google Gemini API for question answering, quiz generation, summarization, and personalized learning recommendations, while the application processes and displays the generated results to the user.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Install FastAPI and Uvicorn: FastfAPI is used to build the backend API and Uvicorn is used to run the application.

```
pip install fastapi uvicorn
```

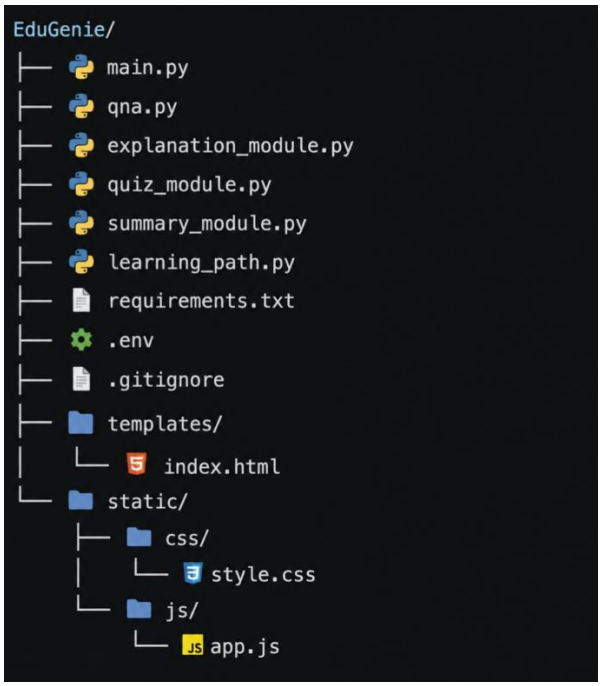
Install Google Gemini SDK: Install the Google GenAI Python SDK to communicate with the Google Gemini API.

```
pip install google-genai
```

Install Python-Dotenv: Install python-dotenv to securely load the Gemini API key from the .env file.

```
pip install python-dotenv
```

Configure the Gemini API Key: Create a .env file in the project root and add:
GEMINI_API_KEY=your_gemini_api_key_here
Set Up Application Structure: Create the EduGenie project structure with the FastAPI backend, AI modules, HTML templates, CSS, JavaScript, environment configuration, and dependency files.



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the core AI-powered educational features of EduGenie using Google Gemini API.

Question & Answer: Allows students to ask questions and receive clear, concise, and student-friendly answers.

Concept Explanation: Explains complex concepts in simple language with examples to improve understanding.

Quiz Generation: Generates educational quizzes based on a selected topic or provided text to help students test their knowledge.

Text Summarization: Converts lengthy educational content into concise and easy-to-understand summaries.

Learning Recommendations: Provides personalized learning recommendations and step-by-step learning paths from beginner to advanced levels.

Activity 2.2: Implement the Flask Backend

Define Routes in FastAPI:
Set up FastAPI routes for the EduGenie home page and AI-powered educational endpoints.
Implement the following endpoints:
/qa
/explain
/quiz
/summarize
/learn/recommendations
Process User Input:
Create request models using Pydantic to receive and validate user input.
Accept student questions, topics, and educational text from the frontend.
Validate the input before sending it for AI processing.
Integrate Google Gemini API:
Connect the backend modules with the Google Gemini API.
Send structured prompts to Gemini based on the selected EduGenie task.
Receive and process the generated AI response.
Return the generated result to the frontend for display.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the EduGenie FastAPI application. In this milestone, the application routes are configured, user requests are validated, the appropriate AI module is selected, and responses are returned to the frontend.

Activity 3.1: Writing the Main Application Logic in main.py

Define the Core Routes in main.py:

Home Page Route: Create a route that renders the main index.html interface.

```
app.get("/"
,
response_cl
ass=@HTMLRe
sponse)
async def
home(reques
```

- **/generate** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

```
app.route("/generate",
methods=["POST"])
def generate():
    data = request.get_json()

    question =
data.get("question",
"").strip()

    if not question:
        return jsonify({
            "error": "Please
enter a question."
        }), 400

    api_key =
os.getenv("GEMINI_API_KEY") or
os.getenv("GOOGLE_API_KEY")

    if not api_key:
        return jsonify({
            "error":
"GEMINI_API_KEY or
GOOGLE_API_KEY is not
configured."
        }), 500

    try:
        client =
genai.Client(api_key=api_key)
```

```
    )

    response_text = response.text.strip()

    return jsonify({
        "success": True,
        "answer": response_text
    })

    except Exception as e:
        import traceback
        traceback.print_exc()

        return jsonify({
            "error": f"Failed to generate answer: {str(e)}"
       }), 500
```

Set Up Route Handlers for Each Feature:

Q&A Route: Read the student's question from the incoming request and send it to the qna.py module for generating an educational answer.
Explanation Route: Receive the topic from the user and call explanation_module.py to provide a simple and understandable explanation.
Quiz Route: Receive the selected topic or text and call quiz_module.py to generate an educational quiz.
Summary Route: Receive the educational passage and call summary_module.py to generate a concise summary.
Learning Recommendation Route: Receive the learner's topic or interest and call learning_path.py to generate a step-by-step learning path.
Input Validation: Validate that the submitted text is not empty and remains within the maximum allowed input length.
Frontend Integration: Handle requests sent from the frontend JavaScript and return the generated AI response to the user interface.

Define Prompt Engineering Helpers:

Educational Prompt Design: Create structured prompts that instruct Google Gemini to provide accurate, clear, and student-friendly educational responses.

```
def build_prompt(question):
    return f"""
You are EduGenie, an educational AI assistant.
Explain the answer clearly using simple,
student-friendly language.

Question: {question}
"""
```

Student-Friendly Responses: Configure prompts to use simple language, headings, bullet points, and examples whenever appropriate.

```
TASKS = {
   "question": "Answer clearly and simply.",
   "explain": "Explain step by step with an example.",
   "quiz": "Create a short quiz with answers.",
   "summary": "Summarize the important points.",
   "recommend": "Suggest useful topics to learn."}
```

Task-Specific Prompts: Use different prompt instructions for Q&A, concept explanation, quiz generation, summarization, and learning recommendations.

```
def build_prompt(question, task="question"):
   instruction = TASKS.get(task, TASKS["question"])
   return f"""
You are EduGenie, an AI learning assistant.
{instruction}
```

Integrate Google Gemini AI Responses in Each Function:

Q&A Integration: Send the student's question to the Google Gemini model and generate clear, accurate, and student-friendly answer.
Explanation Integration: Send the selected topic to the explanation module and use Google Gemini to provide a simplified explanation with examples when useful.
Quiz Integration: Send the selected topic or educational text to Google Gemini and generate a structured quiz for testing the student's understanding.
Summarization Integration: Send the provided educational passage to Google Gemini and generate a concise summary containing the important information.
Learning Recommendation Integration: Send the learner's topic or interest to Google Gemini and generate a step-by-step learning path from beginner to advanced level.

```
def generate_learning_response(question):
    prompt = f"""
You are EduGenie, an AI learning assistant.
Answer the student's question clearly and accurately.
Use simple, student-friendly language.
Explain the concept step by step.
Give an example when useful.
Use headings and bullet points when appropriate.
Do not make the answer unnecessarily long.
Student Question: {question}
"""
    response = client.models.generate_content(
        model="gemini-3.8-flash",
        contents=prompt
    )
    return response.text
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for EduGenie. This involves designing a responsive HTML and CSS interface and using JavaScript to connect the frontend with the FastAPI backend and display AI-generated educational responses.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:
Develop the main HTML file (index.html) as the single-page interface for EduGenie, including the project title, description, task selection, input section, and results section.
Use semantic HTML elements to keep the interface organized and easy to use.

Provide a task selection dropdown containing:

- Question & Answer
- Explanation
- Quiz
- Summary
- Learning Recommendations

Add a text area where students can enter questions, topics, or educational content.
Add a Get Answer button to submit the request.

Design a Responsive Layout Using CSS:
Create the stylesheet at static/css/style.css to provide a clean, modern, and responsive design.
Style the input area, task selection, buttons, loading indicator, result card, and error messages.
Use responsive CSS layouts so the EduGenie interface works properly on desktop, tablet, and mobile screens.

Create UI Components for Each Feature:
Q&A Result: Display the generated answer in a clear result card.
Explanation Result: Display simplified explanations with important points and examples.
Quiz Result: Display the generated quiz questions and answers in an organized format.
Summary Result: Display the summarized educational content in an easy-to-read format.
Learning Recommendation Result: Display the recommended learning path and topics in structured format.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:
Attach a submit event listener to the EduGenie form to prevent the default page refresh and send the user's request to the appropriate FastAPI endpoint using the Fetch API.
Use static/js/app.js to handle the form submission without refreshing the webpage.
Send the selected task and user input to the appropriate FastAPI endpoint using the Fetch API.
During loading, disable the Get Answer button and display a loading indicator.
Render Results Dynamically:
Receive the response from the FastAPI backend.
Display the Google Gemini-generated result inside the result section.
Implement JavaScript functionality to receive the response from the FastAPI backend and dynamically display the generated result in the result section.

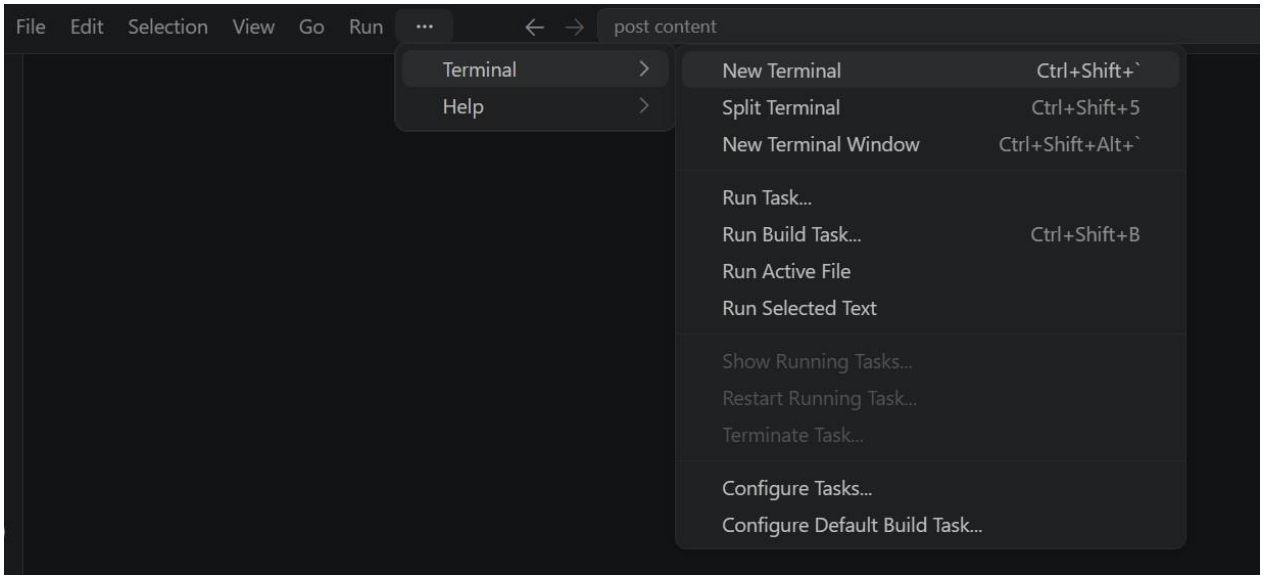
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the EduGenie application locally to verify correct operation. This involves configuring the server environment, managing dependencies, and launching the application for real-world use.

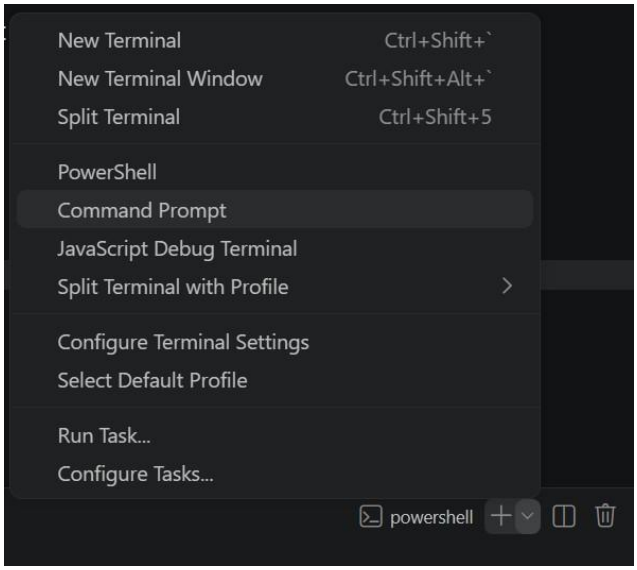
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv

D:\projects>"D:\projects\myenv\Scripts\activate"

(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Gemini API key. The app loads this automatically using python-dotenv at startup:

  GEMINI_API_KEY=your_api_key_here

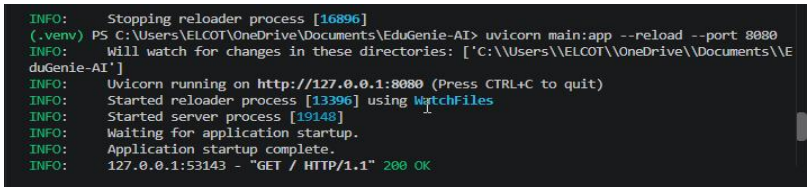
Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>uvicorn main:app --reload
```

- Once running, open a browser and navigate to" http://127.0.0.1:8000 "to access the app.



- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.

Activity 5.3: Public Deployment via Ngrok

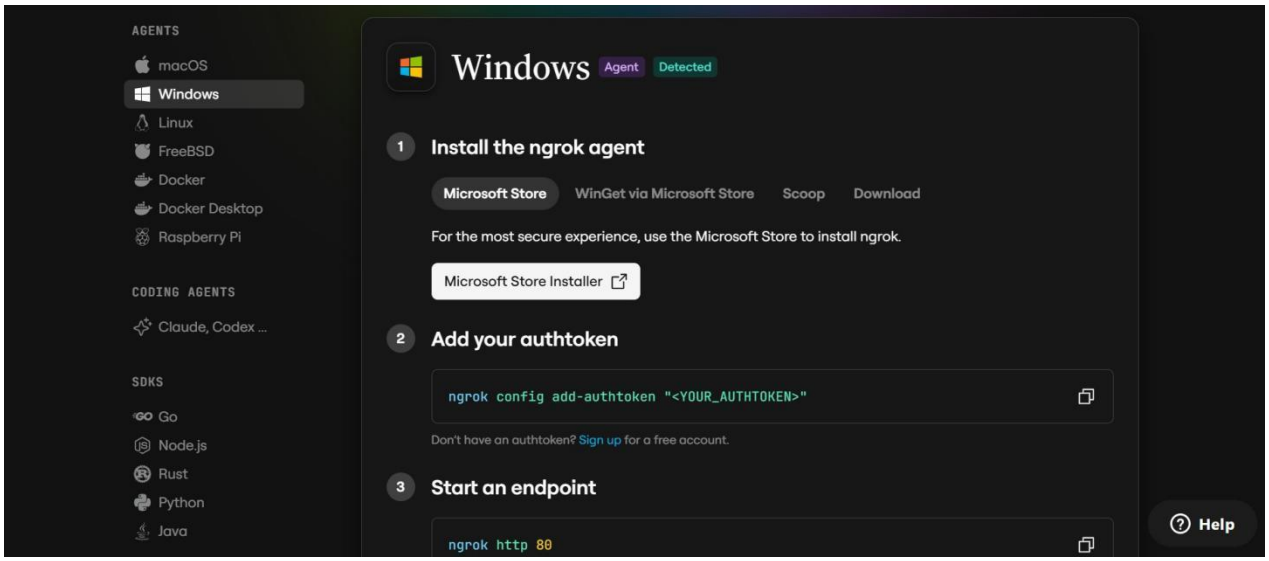
EduGenie can be made accessible through a secure public URL after successfully testing the application locally. The deployment process allows users to access the learning assistant without running the application directly from the development environment.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from https://ngrok.com/download.
- Click on "Microsoft Store Installer".



Configure Deployment Environment:

Ensure all required Python dependencies are installed.

Configure the GEMINI_API_KEY securely using environment variables.

```
GEMINI_API_KEY = "YOUR_API_KEY"
```

Verify that the FastAPI application runs correctly before public deployment.

- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

  ==========

  ======= YOUR PUBLIC WEBSITE URL IS LIVE!

  URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app

  ==========

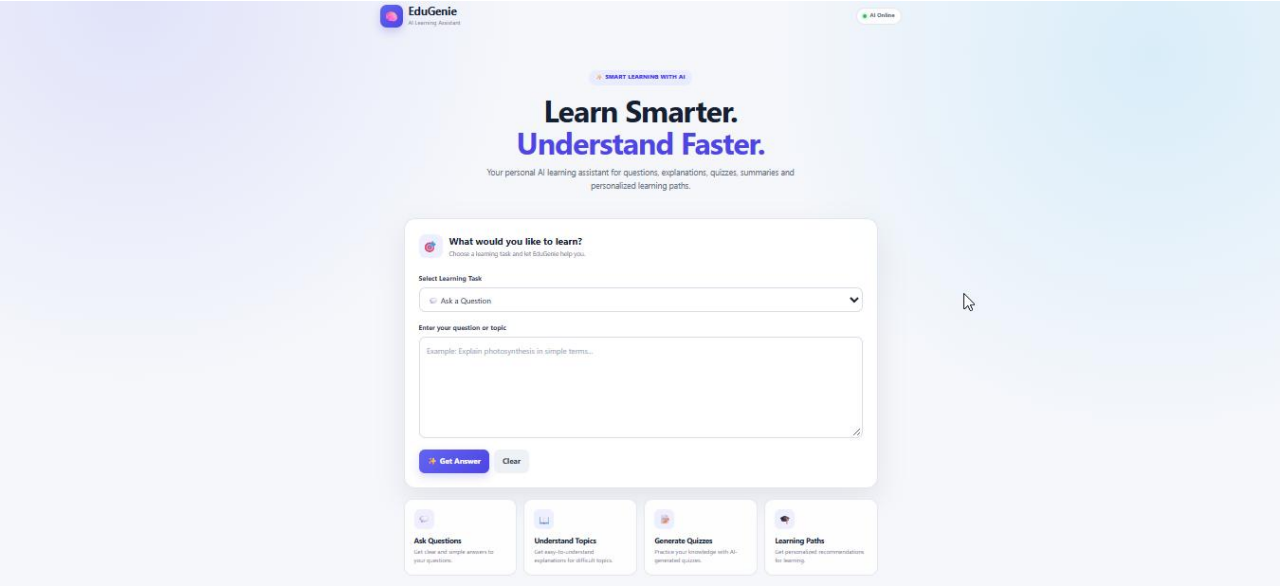
- Share this URL with anyone who can access your CaptionAI app directly from their browser without any installation.

Important Notes:

- ❖ Keep the GEMINI_API_KEY secure and do not expose it in the source code.
- ❖ Make sure the EduGenie server is running before starting the public tunnel.
- ❖ The public URL may change when the tunnel is restarted.
- ❖ Test the application locally before making it publicly accessible.

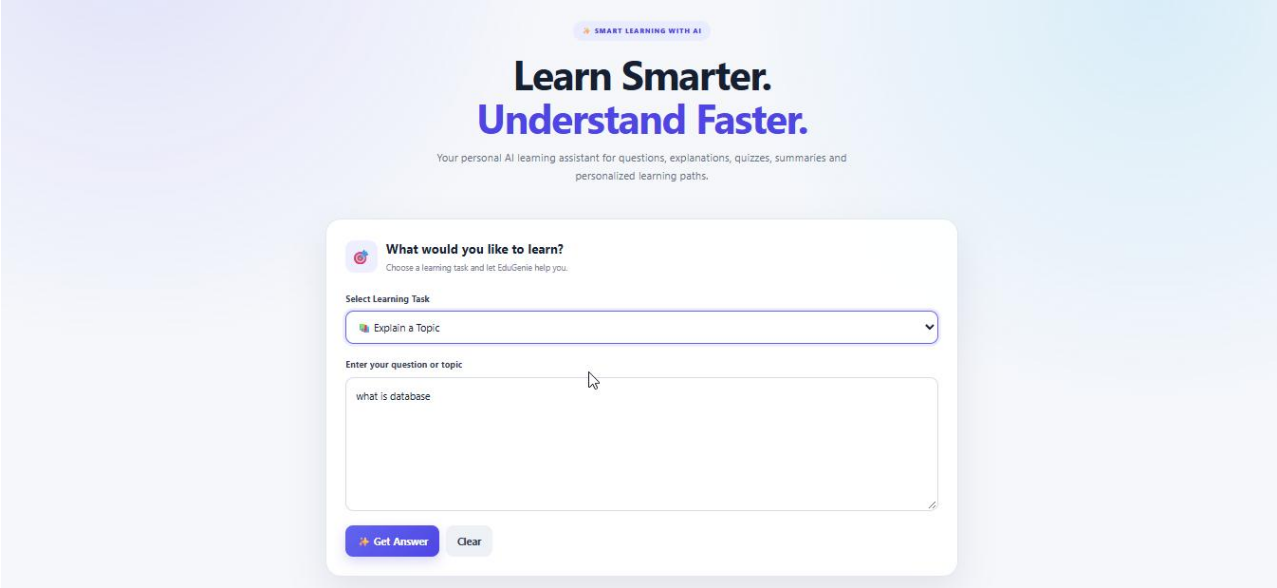
Exploring the Website's Web Pages:

Home / Generator Page:



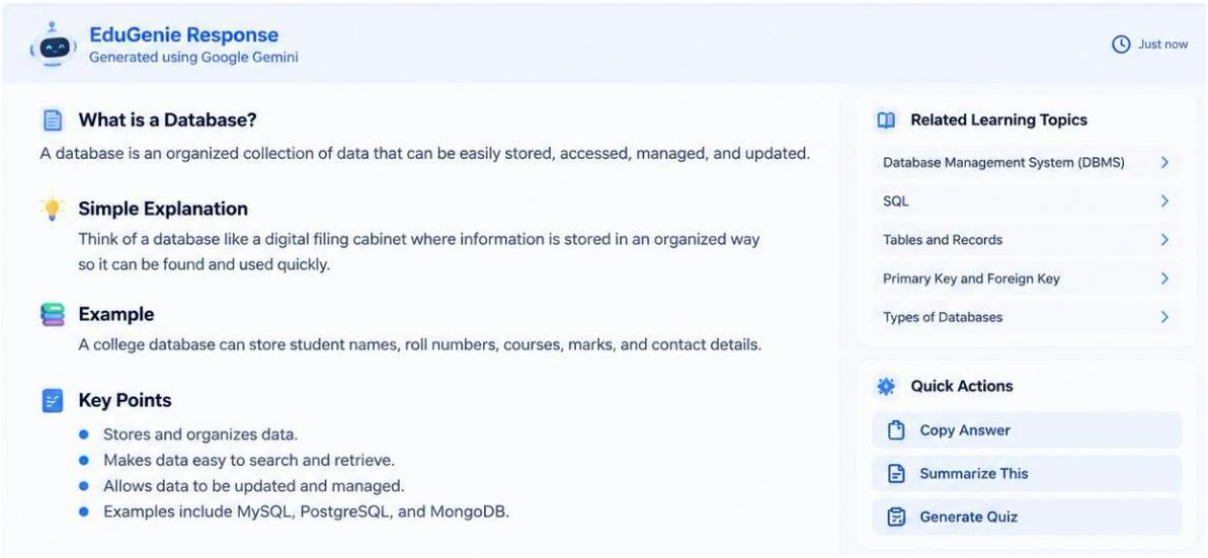
Description: The EduGenie single-page interface serves as the main learning assistant page. It provides students with an easy-to-use interface for asking educational questions, receiving clear Al-generated explanations, generating quizzes, summarizing learning content, and creating personalized learning plans. The page presents the EduGenie branding, question input area, feature options, and Al-generated response section in one seamless interface.

Input Section:



Description: The EduGenie single-page interface serves as the main learning assistant page. It provides students with an easy-to-use interface for asking educational questions, receiving clear Al-generated explanations, generating quizzes, summarizing learning content, and creating personalized learning plans. The page presents the EduGenie branding, question input area, feature options, and Al-generated response section in one seamless interface.

Results Section:



Description: The EduGenie single-page interface serves as the main learning assistant page. It provides students with an easy-to-use interface for asking educational questions, receiving clear Al-generated explanations, generating quizzes, summarizing learning content, and creating personalized learning plans. The page presents the EduGenie branding, question input area, feature options, and Al-generated response section in one seamless interface.

Conclusion:

EduGenie: Google Gemini Powered Learning Assistant is an AI-based educational platform designed to support students with personalized and interactive learning. It uses Google Gemini to provide clear answers, explanations, summaries, quizzes, and learning recommendations based on student queries. The project focuses on making learning easier, faster, and more accessible through a simple user-friendly interface. With further improvements such as personalized learning paths, expanded educational resources, and additional AI-powered features, EduGenie can evolve into a comprehensive learning assistant that supports students in their academic journey.